Guide to using dplyr

We'll be covering the following functions:

- filter() (and slice())
- arrange()
- select() (and rename())
- distinct()
- mutate() (and transmute())
- summarise()
- sample_n() and sample_frac()

Installing

You can install **dplyr** using

In [ ]:

```
install.packages('dplyr')
```

In [12]:

```
# Run it using
library(dplyr)
```

Example Data

Let's use some flight data for our examples. We'll download the nycflights13 data package:

In [ ]:

```
install.packages('nycflights13',repos = 'http://cran.us.r-project.org')
```

In [26]:

```
library(nycflights13)
summary(flights)
```

Out[26]:

```
      year          month             day          dep_time    sched_dep_time
 Min.   :2013   Min.   : 1.000   Min.   : 1.00   Min.   :   1   Min.   : 106
 1st Qu.:2013   1st Qu.: 4.000   1st Qu.: 8.00   1st Qu.: 907   1st Qu.: 906
 Median :2013   Median : 7.000   Median :16.00   Median :1401   Median :1359
 Mean   :2013   Mean   : 6.549   Mean   :15.71   Mean   :1349   Mean   :1344
 3rd Qu.:2013   3rd Qu.:10.000   3rd Qu.:23.00   3rd Qu.:1744   3rd Qu.:1729
 Max.   :2013   Max.   :12.000   Max.   :31.00   Max.   :2400   Max.   :2359
                                                 NA's   :8255
   dep_delay          arr_time    sched_arr_time   arr_delay
 Min.   : -43.00   Min.   :   1   Min.   :   1   Min.   : -86.000
 1st Qu.:  -5.00   1st Qu.:1104   1st Qu.:1124   1st Qu.: -17.000
 Median :  -2.00   Median :1535   Median :1556   Median :  -5.000
 Mean   :  12.64   Mean   :1502   Mean   :1536   Mean   :   6.895
 3rd Qu.:  11.00   3rd Qu.:1940   3rd Qu.:1945   3rd Qu.:  14.000
 Max.   :1301.00   Max.   :2400   Max.   :2359   Max.   :1272.000
 NA's   :8255      NA's   :8713                  NA's   :9430
   carrier             flight       tailnum             origin
 Length:336776      Min.   :   1   Length:336776      Length:336776
 Class :character   1st Qu.: 553   Class :character   Class :character
 Mode  :character   Median :1496   Mode  :character   Mode  :character
                    Mean   :1972
                    3rd Qu.:3465
                    Max.   :8500
```

```
    dest              air_time        distance         hour
 Length:336776      Min.   : 20.0   Min.   :  17   Min.   : 1.00
 Class :character   1st Qu.: 82.0   1st Qu.: 502   1st Qu.: 9.00
 Mode  :character   Median :129.0   Median : 872   Median :13.00
                    Mean   :150.7   Mean   :1040   Mean   :13.18
                    3rd Qu.:192.0   3rd Qu.:1389   3rd Qu.:17.00
                    Max.   :695.0   Max.   :4983   Max.   :23.00
                    NA's   :9430
     minute        time_hour
 Min.   : 0.00   Min.   :2013-01-01 05:00:00
 1st Qu.: 8.00   1st Qu.:2013-04-04 13:00:00
 Median :29.00   Median :2013-07-03 10:00:00
 Mean   :26.23   Mean   :2013-07-03 05:02:36
 3rd Qu.:44.00   3rd Qu.:2013-10-01 07:00:00
 Max.   :59.00   Max.   :2013-12-31 23:00:00
```

In [27]:

```
# Notice how large the data frame is:
dim(flights)
```

Out[27]:

336776 19

filter()

filter() allows you to select a subset of rows in a data frame. The first argument is the name of the data frame. The second and subsequent arguments are the expressions that filter the data frame:

For example, we can select all flights on November 3rd that were from American Airlines (AA) with:

In [32]:

```
head(filter(flights,month==11,day==3,carrier=='AA'))
```

Out[32]:

	year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time	arr_delay	carrier	flight	tailnur
1	2013	11	3	538	545	-7	824	855	-31	AA	2243	N5DWA
2	2013	11	3	556	600	-4	900	905	-5	AA	1175	N3CSA
3	2013	11	3	604	610	-6	844	855	-11	AA	1103	N3KDA
4	2013	11	3	624	629	-5	907	929	-22	AA	1205	N3EJA
5	2013	11	3	625	630	-5	736	805	-29	AA	303	N4WJA
6	2013	11	3	653	655	-2	925	920	5	AA	1263	N634A

This is a lot simpler than the normal way to do this with a dataframe:

In [36]:

```
head(flights[flights$month == 11 & flights$day == 3 & flights$carrier == 'AA', ])
```

Out[36]:

	year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time	arr_delay	carrier	flight	tailnur
1	2013	11	3	538	545	-7	824	855	-31	AA	2243	N5DWA
2	2013	11	3	556	600	-4	900	905	-5	AA	1175	N3CSA
3	2013	11	3	604	610	-6	844	855	-11	AA	1103	N3KDA
4	2013	11	3	624	629	-5	907	929	-22	AA	1205	N3EJA
5	2013	11	3	625	630	-5	736	805	-29	AA	303	N4WJA
6	2013	11	3	653	655	-2	925	920	5	AA	1263	N634A

slice()

We can select rows by position using **slice()**

In [37]:

```
slice(flights, 1:10)
```

Out[37]:

	year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time	arr_delay	carrier	flight	tailnu
1	2013	1	1	517	515	2	830	819	11	UA	1545	N142
2	2013	1	1	533	529	4	850	830	20	UA	1714	N242
3	2013	1	1	542	540	2	923	850	33	AA	1141	N619
4	2013	1	1	544	545	-1	1004	1022	-18	B6	725	N804
5	2013	1	1	554	600	-6	812	837	-25	DL	461	N668
6	2013	1	1	554	558	-4	740	728	12	UA	1696	N394
7	2013	1	1	555	600	-5	913	854	19	B6	507	N516

	year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time	arr_delay	carrier	flight	tailnu
8	2013	1	1	557	600	-3	709	723	-14	EV	5708	N829A
9	2013	1	1	557	600	-3	838	846	-8	B6	79	N593.
10	2013	1	1	558	600	-2	753	745	8	AA	301	N3AL/

arrange()

arrange() works similarly to filter() except that instead of filtering or selecting rows, it reorders them. It takes a data frame, and a set of column names (or more complicated expressions) to order by. If you provide more than one column name, each additional column will be used to break ties in the values of preceding columns:

In [40]:

```
head(arrange(flights,year,month,day,air_time))
```

Out[40]:

	year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time	arr_delay	carrier	flight	tailnur
1	2013	1	1	2302	2200	62	2342	2253	49	EV	4276	N13903
2	2013	1	1	1318	1322	-4	1358	1416	-18	EV	4106	N19554
3	2013	1	1	2116	2110	6	2202	2212	-10	EV	4404	N15912
4	2013	1	1	2000	2000	0	2054	2110	-16	9E	3664	N836A
5	2013	1	1	2056	2004	52	2156	2112	44	EV	4170	N1254(
6	2013	1	1	908	915	-7	1004	1033	-29	US	1467	N959U

You can add desc() to arrange in descending order:

In [42]:

```
head(arrange(flights,desc(dep_delay)))
```

Out[42]:

	year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time	arr_delay	carrier	flight	tailnur
1	2013	1	9	641	900	1301	1242	1530	1272	HA	51	N384H/

	year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time	arr_delay	carrier	flight	tailnur
2	2013	6	15	1432	1935	1137	1607	2120	1127	MQ	3535	N504M
3	2013	1	10	1121	1635	1126	1239	1810	1109	MQ	3695	N517M
4	2013	9	20	1139	1845	1014	1457	2210	1007	AA	177	N338A
5	2013	7	22	845	1600	1005	1044	1815	989	MQ	3075	N665M
6	2013	4	10	1100	1900	960	1342	2211	931	DL	2391	N959D

select()

Often you work with large datasets with many columns but only a few are actually of interest to you. select() allows you to rapidly zoom in on a useful subset using operations that usually only work on numeric variable positions:

In [43]:

```
head(select(flights,carrier))
```

Out[43]:

	carrier
1	UA
2	UA
3	AA
4	B6
5	DL
6	UA

rename()

You can use rename() to rename columns, note this is not "in-place" you'll need to reassign the renamed data structures.

In [44]:

```
head(rename(flights,airline_car = carrier))
```

Out[44]:

	year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time	arr_delay	airline_car	flight	tai
1	2013	1	1	517	515	2	830	819	11	UA	1545	N1
2	2013	1	1	533	529	4	850	830	20	UA	1714	N2
3	2013	1	1	542	540	2	923	850	33	AA	1141	N6

	year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time	arr_delay	airline_car	flight	tail
4	2013	1	1	544	545	-1	1004	1022	-18	B6	725	N6
5	2013	1	1	554	600	-6	812	837	-25	DL	461	N6
6	2013	1	1	554	558	-4	740	728	12	UA	1696	N3

distinct()

A common use of select() is to find the values of a set of variables. This is particularly useful in conjunction with the distinct() verb which only returns the unique values in a table.

In [46]:

```
distinct(select(flights,carrier))
```

Out[46]:

	carrier
1	UA
2	AA
3	B6
4	DL
5	EV
6	MQ
7	US
8	WN
9	VX
10	FL
11	AS
12	9E
13	F9
14	HA
15	YV
16	OO

mutate()

Besides selecting sets of existing columns, it's often useful to add new columns that are functions of existing columns. This is the job of mutate():

In [50]:

```
head(mutate(flights, new_col = arr_delay-dep_delay))
```

Out[50]:

	year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time	arr_delay	carrier	flight	tailnur
1	2013	1	1	517	515	2	830	819	11	UA	1545	N14228

	year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time	arr_delay	carrier	flight	tailnur
2	2013	1	1	533	529	4	850	830	20	UA	1714	N2421
3	2013	1	1	542	540	2	923	850	33	AA	1141	N619A
4	2013	1	1	544	545	-1	1004	1022	-18	B6	725	N804JE
5	2013	1	1	554	600	-6	812	837	-25	DL	461	N668D
6	2013	1	1	554	558	-4	740	728	12	UA	1696	N3946

transmute()

Use transmute if you only want the new columns:

In [52]:

```
head(transmute(flights, new_col = arr_delay-dep_delay))
```

Out[52]:

	new_col
1	9
2	16
3	31
4	-17
5	-19
6	16

summarise()

You can use summarise() to quickly collapse data frames into single rows using functions that aggregate results. Remember to use na.rm=TRUE to remove NA values.

In [63]:

```
summarise(flights,avg_air_time=mean(air_time,na.rm=TRUE))
```

Out[63]:

	avg_dep_delay
1	150.6865

sample_n() and sample_frac()

You can use sample_n() and sample_frac() to take a random sample of rows: use sample_n() for a fixed number and sample_frac() for a fixed fraction.

In [55]:

```
sample_n(flights,10)
```

Out[55]:

	year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time	arr_delay	carrier	flight	tailnu
1	2013	3	5	833	838	-5	937	957	-20	UA	367	N405U
2	2013	1	31	804	810	-6	1052	1044	8	FL	346	N922A
3	2013	10	25	1955	1945	10	2201	2143	18	EV	5306	N738E
4	2013	5	24	2202	2100	62	2343	2253	50	EV	4700	N155
5	2013	4	5	1802	1800	2	2140	2140	0	AA	177	N323A
6	2013	4	4	634	635	-1	731	755	-24	UA	1142	N754
7	2013	8	27	1716	1720	-4	1927	1920	7	MQ	3556	N502I
8	2013	4	26	1155	1200	-5	1248	1315	-27	9E	3341	N905X
9	2013	9	23	1906	1908	-2	2210	2220	-10	UA	1515	N167
10	2013	8	2	959	1003	-4	1132	1135	-3	FL	353	N992A

In [60]:

```
# .005% of the data
sample_frac(flights,0.00005) # USE replace=TRUE for bootstrap sampling
```

Out[60]:

	year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time	arr_delay	carrier	flight	tailnu
1	2013	7	30	649	643	6	855	855	0	EV	4393	N121
2	2013	4	19	2052	1850	122	120	2136	224	B6	527	N651
3	2013	9	18	740	655	45	843	809	34	B6	318	N183
4	2013	11	16	559	600	-1	928	927	1	UA	665	N534U

	year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time	arr_delay	carrier	flight	tailnu
5	2013	10	4	559	600	-1	710	715	-5	WN	464	N917
6	2013	3	11	1500	1505	-5	1813	1822	-9	UA	1178	N775
7	2013	1	21	1653	1655	-2	1945	2010	-25	B6	185	N703
8	2013	8	8	28	2305	83	125	13	72	B6	718	N329
9	2013	10	23	843	700	103	1136	1015	81	VX	399	N626
10	2013	4	24	1804	1800	4	1956	1950	6	AA	353	N564
11	2013	10	10	1614	1620	-6	1741	1755	-14	DL	2443	N336
12	2013	9	20	1155	1200	-5	1303	1334	-31	UA	255	N821
13	2013	8	21	1259	1300	-1	1521	1519	2	EV	5148	N176
14	2013	2	1	2120	2125	-5	2233	2250	-17	MQ	4660	N508
15	2013	2	19	1548	1526	22	1810	1801	9	UA	647	N567
16	2013	1	4	835	841	-6	1107	1113	-6	EV	4388	N139
17	2013	9	2	834	840	-6	1011	1007	4	EV	4594	N229

Conclusion

Hopefully you've seen how dplyr() can save you lots of time and headaches! Remember to use **help()** or just reference the documentation if you ever need help using the package!